

What Does Content-Blindness Cost? Scheduling LLM Inference Without Reading the Prompt

Anonymous Author(s)

Affiliation withheld for double-blind review

Abstract—Large language model (LLM) inference services schedule requests without knowing how many tokens each will generate. Because a long response occupies a batch slot for many iterations, first-come-first-serve (FCFS) scheduling suffers head-of-line blocking, and a substantial literature therefore predicts output length from the prompt text in order to approximate shortest-job-first. Reading prompt content is precisely what a privacy-constrained or regulated serving platform cannot do. We quantify what that restriction costs. Analysing 44.1 million requests from Microsoft Azure’s production LLM inference traces, we observe that service work splits into a prefill component, which is exactly observable at admission from the context length alone, and a decode component, which is not observable until the request completes, and that the observable component is the larger one: prefill is 91.2% and 98.7% of attributable marginal work, and 76% and 92% of measured engine time at high load. We evaluate CB-SJF-Work, a scheduler that orders admissions by estimated total service work using only token counts and arrival metadata. Over 20 (conversation) and 11 (code) windows sampled across a full production week, it reduces mean normalised latency by 51.5% and 58.8% relative to FCFS and recovers 90.3% and 79.3% of the improvement attainable by an oracle with perfect knowledge of output length. Two findings qualify this. First, the benefit is not free: shortest-first ordering defers long requests, and 99th-percentile latency degrades by up to $1.9\times$ (conversation) and $3.1\times$ (code) at the highest load, a cost that an oracle scheduler incurs equally. Second, a learned content-blind output-length predictor contributes almost nothing over ordering by raw context length (0.1% and 1.3%), and ordering by predicted output length alone is worse than FCFS on the code workload. For these workloads the information a scheduler is denied turns out to be the information it least needs.

Index Terms—LLM inference serving, cloud scheduling, head-of-line blocking, privacy-preserving systems, production trace analysis, big data architectures

I. INTRODUCTION

Serving large language models is now a dominant consumer of cloud GPU capacity. A modern inference engine such as vLLM [1] or Orca [2] executes requests with *continuous batching*: many requests share a batch, each scheduling iteration emits one token for every resident request, and a request holds its slot and its share of the key-value (KV) cache until it produces its final token.

This creates a scheduling problem with an unusual information structure. How long a request holds its slot is proportional to its output length, and that length is unknown when the request arrives. Under first-come-first-serve (FCFS), the default in production engines, one long-generating request admitted early blocks many short ones behind it, producing head-of-line blocking and long tails.

The established response is to predict output length and approximate shortest-job-first (SJF). S^3 fine-tunes a Distil-BERT model on the prompt to predict sequence length [3]; Fu *et al.* learn to rank requests by predicted output length, again from prompt text [4]. Both report large gains, and both require the scheduler to read the user’s prompt. That is a genuine obstacle: a platform under data-protection obligations, serving regulated sectors, or offering confidential-computing guarantees may be unable to route prompt content into a scheduling component, and running an auxiliary transformer over every prompt consumes the accelerator capacity the scheduler is trying to conserve. Hence our question: how much of the achievable scheduling benefit remains available to a scheduler that never reads prompt content?

We answer it empirically on Microsoft Azure’s public production LLM inference traces [5], which contain 44.1 million real requests across a conversation workload and a code workload. Our central finding is that the question’s premise, that the valuable information is locked inside the prompt, does not hold for these workloads. Service time has two components: a prefill pass over the input tokens, whose cost is fixed by the context length and therefore visible at admission without reading any content, and a sequence of decode iterations, whose cost is not. On these traces prefill is the larger component under either of the two accountings we report in Section III.

We make the following contributions.

- 1) A quantification of what content-blindness costs an admission scheduler. Ordering by observable work recovers 90.3% (conversation) and 79.3% (code) of the improvement an output-length oracle attains, evaluated over 20 and 11 windows sampled across a full production week (Sections V–VI).
- 2) A characterisation of the observable and unobservable components of service work on 44.1M production requests, reported under two accountings so the conclusion does not rest on one cost convention, together with the finding that output length is weakly predictable from content-blind metadata for conversational traffic ($\rho_s = 0.365$) and effectively unpredictable for code completion ($\rho_s = 0.107$) (Section III).
- 3) Two negative results. A learned content-blind length predictor adds almost nothing over ordering by raw context length, and ordering by predicted output length alone is worse than FCFS on the code workload. Under a content-blindness constraint, the effort of learning to

predict output length is not repaid.

- 4) An accounting of the cost side. Every work-aware or length-aware policy, including both oracles, degrades tail latency relative to FCFS. We report the magnitude rather than the mean-latency gain alone.

II. BACKGROUND AND RELATED WORK

Continuous-batching inference engines. Orca [2] introduced iteration-level scheduling, and vLLM [1] added paged KV-cache management, together establishing the execution model we assume: a batch whose membership changes every iteration, bounded by GPU memory available for the KV cache. Sarathi-Serve [6] reduces prefill-induced decode stalls by chunking prefills, and Splitwise [7] and DistServe [8] disaggregate the two phases onto separate hardware. These works change where and how prefill and decode execute; they are orthogonal to, and composable with, the question of which request to admit next, which is our subject.

Output-length prediction for scheduling. S^3 [3] predicts output length with a fine-tuned DistilBERT to reserve KV memory more tightly. Fu *et al.* [4] observe that exact lengths are unnecessary and learn only the relative rank of output lengths, reporting substantial latency reductions. Both extract their signal from prompt text, which is the constraint we remove.

Scheduling on input length. We are not the first to key scheduling on input length. FastServe’s skip-join MLFQ [9] assigns each arrival an initial queue from its profiled prefill time, and absorbs the unknown output length through iteration-level preemption rather than prompt inspection. PecSched [10] analyses an Azure LLM inference trace, reports the same skewed input-length distribution we observe, and preemptively prioritises short-input requests at cluster scale. Both are prior art for content-blind, input-length-keyed scheduling, and both obtain their tail behaviour from preemption, which our non-preemptive setting excludes. Our question is narrower: given that output length is unobservable, how much of the oracle-attainable benefit does a scheduler forfeit?

Production trace analysis. Characterising real cloud workloads has repeatedly overturned assumptions formed on synthetic ones, as with Resource Central for VM allocation [11]. The Azure LLM inference traces [5] were released alongside Splitwise [7] and DynamoLLM [12]. Splitwise already reports that the coding workload is prompt-dominated, with a median of 13 generated tokens, and we do not claim that observation as new. What we add is its consequence for admission scheduling: how the observable and unobservable components of service work divide, and what a scheduler forfeits by seeing only the former.

III. CHARACTERISING PRODUCTION LLM TRAFFIC

A. Traces

We use the Azure LLM Inference Trace 2024 [5], released under CC-BY-4.0/MIT. It contains two one-week traces from Microsoft’s production inference service: a conversation workload (27.3M requests) and a code workload (16.8M requests).

Each record gives an arrival timestamp, the input context length in tokens, and the number of tokens generated. Crucially for our purpose, the traces contain no prompt text: they are themselves a content-blind view of the workload, which is exactly the information a privacy-constrained scheduler would possess.

B. The observable component is the larger one

In a continuous-batching engine, servicing a request costs a prefill pass proportional to its context length x_i , plus y_i decode iterations where y_i is its output length. Charging decode at its marginal per-token cost (the per-iteration base cost is a property of the batch, not of any one request), the work attributable to request i is

$$W_i = c_f + c_p x_i + c_d y_i, \quad (1)$$

where c_p is the per-token prefill cost, c_d the marginal per-token decode cost, and c_f a fixed overhead. The first two terms are known at admission; only the third requires knowing y_i .

Evaluated over both full traces, (1) puts prefill at 91.2% of attributable marginal work in the conversation trace and 98.7% in the code trace. The reason is the traffic shape. Mean context lengths are 1632 and 2511 tokens against mean output lengths of only 106 and 23, giving aggregate context-to-output token ratios of 15.5:1 and 110.7:1. That ratio is a property of the traffic and involves no cost model at all.

Equation (1) is an attribution convention, not a measurement of engine time, because it charges decode only its marginal cost and assigns the per-iteration base cost to the batch rather than to any request. Since our argument rests on this split, we also report the other natural accounting: the fraction of busy time our simulated engine actually spends in prefill steps. That fraction is load-dependent, because larger batches amortise the base decode cost over more requests. At $\rho = 0.60$ it is 53.9% (conversation) and 81.8% (code); at $\rho = 0.92$ it is 75.6% and 92.4%. The two accountings differ, most visibly for conversation at low load, and we report both rather than the more favourable one. Under either, prefill is at least half of the work and the component hidden from the scheduler is the smaller one.

C. Output length is not equally predictable

The unobservable component is also unequally guessable. Binning each trace by context-length decile, mean output length rises roughly ninefold across the deciles of the conversation trace, so context length carries real information about output length there. For code completion the relationship is flat and output length is statistically independent of input length. A content-blind predictor (Section IV) attains a rank correlation of $\rho_s = 0.365$ with true output length on conversation but only 0.107 on code, with pairwise ordering accuracies of 0.633 and 0.537 against a chance level of 0.5.

The two workloads therefore have opposite risk profiles. The code trace has both the heavier output-length tail and the weaker predictive signal: it is the workload that would benefit

most from length-aware scheduling and the one where predicting length is hardest. A method that depends on accurate length prediction is weakest exactly where it is most needed.

IV. CONTENT-BLIND WORK-AWARE SCHEDULING

A. Threat model and information constraint

The scheduler may observe, per arriving request, the context length in tokens, the arrival timestamp, the workload class, and aggregate statistics over requests that have already completed. It may not observe prompt or response text. This corresponds to a front end that tokenises and meters requests, which it must do for billing regardless, but forwards content only to the model.

B. Policy

Section III implies that ordering by predicted output length optimises the wrong quantity. What determines contention is total service work W_i , and by (1) the observable terms dominate it. CB-SJF-Work therefore admits requests in increasing order of

$$\widehat{W}_i = c_f + c_p x_i + c_d \hat{y}_i, \quad (2)$$

where $\hat{y}_i$ is a content-blind estimate of output length. Because $c_d \hat{y}_i$ contributes only a small share of $\widehat{W}_i$, the policy degrades gracefully: when $\hat{y}_i$ is uninformative, the ranking is still governed by the exactly-known prefill term.

We obtain $\hat{y}_i$ from a gradient-boosted regression tree trained on $\log(1 + y)$ over ten content-blind features: context length and its logarithm, workload class, hour of day, day of week, inter-arrival time, recent arrival rate, and rolling mean and 90th-percentile output length and mean context length over the previous 512 completed requests. Rolling statistics are shifted by one request, so no request observes its own or any future outcome, and an automated check fails the run if any feature exceeds 0.95 rank correlation with the target. The predictor trains on a contiguous earlier period of the same trace, disjoint from every evaluation window, and is applied unchanged thereafter.

Admission is head-of-line: if the highest-priority waiting request does not fit in the KV cache, the scheduler waits rather than scanning past it. The KV budget for a model with L layers, H KV heads and head dimension d at b bytes per element is

$$B = \lfloor (M_{\text{gpu}} - M_w) / (2LHdb) \rfloor \quad (3)$$

tokens, where M_w is the weight footprint. Note that the KV reservation in Algorithm 1 charges the true output length y_i , not the estimate $\hat{y}_i$. This is a simplification: reserving on an underestimate would require the preemption and recompute path that real engines use when a reservation is exceeded, which we do not model. It applies identically under every policy, including FCFS and both oracles, so it confers no advantage on any of them. The scheduler’s ordering key, which is what this paper studies, remains strictly content-blind.

Algorithm 1 CB-SJF-Work admission (one scheduling iteration)

Require: pending min-heap Q , running set R , KV budget B , used u

- 1: **on arrival of** i : push i into Q with key $\widehat{W}_i$ from (2) $\triangleright$ content-blind
- 2: **while** $Q \neq \emptyset$ **do**
- 3: $i \leftarrow \text{PEEK}(Q)$
- 4: **if** $|R| < B_{\text{max}}$ **and** $u + x_i + y_i \leq B$ **then**
- 5: $\text{POP}(Q)$; $R \leftarrow R \cup \{i\}$; $u \leftarrow u + x_i + y_i$
- 6: **else**
- 7: **break** $\triangleright$ head-of-line; do not scan past
- 8: **end if**
- 9: **end while**
- 10: run prefill for newly admitted requests, else one decode iteration
- 11: evict completed requests from R and release their KV

V. EXPERIMENTAL METHODOLOGY

Simulator. We built a discrete-event simulator of one continuous-batching replica with iteration-level scheduling and the KV bound of (3). Prefill cost is affine in prompt tokens, decode cost affine in batch size, with parameters for a 13B-parameter model ($L = 40$, $H = 40$, $d = 128$, fp16) on an 80 GB accelerator, giving $B = 65,917$ tokens. The cost model is analytical, not measured on our hardware; Section VII reports a sensitivity analysis.

Load calibration. The published traces are fleet-aggregate (75–87 req/s), far above single-replica capacity. We therefore measure each window’s saturated throughput μ by driving the engine with an unbounded backlog, then rescale the arrival timeline so that $\rho = \lambda/\mu$ takes the values $\{0.60, 0.75, 0.85, 0.92, 0.98\}$. This preserves the arrival process shape and request mix up to an affine rescaling of the clock. Measured capacity is 1.79 req/s (conversation) and 1.41 req/s (code).

Two caveats follow, because ρ is easy to misread. First, μ is measured at maximum concurrency, where the per-iteration base cost amortises over a full batch, whereas the runs operate at smaller batches; measured utilisation therefore exceeds ρ substantially, reaching 0.91 (conversation) and 0.71 (code) at $\rho = 0.60$. Read ρ as a control variable, not as utilisation. Second, this does not bias the comparison: throughput is identical across policies to four significant figures.

Windows. We evaluate the first 20,000 requests of each of 20 (conversation) and 11 (code) windows sampled at even intervals across the full week, each at least two hours from the next and after the predictor’s training period. Twenty thousand requests span roughly 4 to 13 minutes of arrivals, so a window is a few minutes of traffic. We do not select busy windows: because arrivals are rescaled to a target ρ , a window’s absolute arrival rate is normalised away and window choice affects only the request mix, so sampling across the week costs nothing and covers diurnal variation. Replaying the same windows under

every policy makes all comparisons paired. The leading 10% of each run is discarded as warm-up.

Policies. We compare FCFS (the engine default); SJF-Ctx, which orders by context length alone; SJF-Pred, which orders by predicted output length (the objective of prior work, realised content-blind); our CB-SJF-Work; and two oracles: Oracle-SJF, ordering by true output length, and Oracle-Work, ordering by true total work. The oracles require information unavailable at admission and are upper bounds, not deployable systems.

Metrics. Mean normalised latency $\bar{L}_n$ (end-to-end latency per output token, the standard continuous-batching metric), 99th-percentile end-to-end latency, and SLO attainment, defined as the fraction of requests meeting both a 2 s time-to-first-token and a 100 ms time-between-tokens target.

VI. RESULTS

A. Content-blind scheduling captures most of the benefit

Table I reports the contended operating point $\rho = 0.92$. CB-SJF-Work reduces mean normalised latency from 1161 to 506 (conversation) and from 16724 to 6327 (code). Fig. 1 shows the full load sweep. Below $\rho \approx 0.75$ all policies nearly coincide, because the admission queue is shallow and there is little ordering decision to make; we report this rather than restricting the evaluation to loads that flatter the method. Above that point the policies separate.

Table II quantifies how much of the attainable benefit survives the content-blind restriction. Averaged over $\rho \geq 0.85$, CB-SJF-Work recovers 90.3% of the Oracle-Work improvement on the conversation workload and 79.3% on code. The improvement is positive in every window at every load above $\rho = 0.75$ (one-sided Wilcoxon signed-rank, $p < 10^{-3}$).

Two cautions on reading these percentages. The headline improvement is a ratio of means, so it is weighted by whichever window has the worst FCFS run; the per-window range in Table II is wide (6% to 77% on conversation at $\rho \geq 0.85$) and the mean of per-window improvements is 47.0%, not 51.5%. Second, the recovered-gap percentage is a ratio of two quantities that both approach zero at low load, so its high value at $\rho = 0.60$ describes a setting where the absolute improvement is under 1% and should not be read as success.

B. The mean-latency gain is paid for in the tail

Shortest-first ordering reduces the mean by deferring long requests, so it must lengthen the tail. We report that cost rather than leave it to be inferred from Table I. On code, CB-SJF-Work has a worse 99th-percentile latency than FCFS in 31 of 33 window-load cells at $\rho \geq 0.85$, by a mean factor of $2.05\times$, rising to $3.12\times$ at $\rho = 0.98$. On conversation the tail is essentially unchanged up to $\rho = 0.92$ ($0.98\times$ and $1.00\times$) and degrades to $1.86\times$ at $\rho = 0.98$.

This cost is not attributable to content-blindness. Oracle-Work incurs it equally, with P_{99} ratios against FCFS of $1.42\times$, $1.95\times$ and $3.30\times$ on code at the three highest loads, slightly worse than ours, and Oracle-SJF is worse still (1261 s

against our 556 s at $\rho = 0.92$). The penalty is a property of shortest-first admission, not of the information the scheduler is denied. An operator who cares primarily about the tail should not deploy any of these policies without an anti-starvation mechanism; preemptive schedulers [9], [10] address exactly this.

C. The learned predictor earns almost nothing

CB-SJF-Work differs from SJF-Ctx only by the $c_d \hat{y}_i$ term. Averaged over $\rho \geq 0.85$ that term is worth 0.12% on conversation and 1.30% on code, and is statistically indistinguishable from zero in every cell except code at $\rho = 0.98$ (one-sided Wilcoxon, $p = 0.009$, positive in 9 of 11 windows, worth 3.45%). At $\rho = 0.92$ the two differ by 506 against 505 (conversation) and 6327 against 6337 (code). Ordering by raw context length is, for practical purposes, as good as ordering by a learned estimate of total work, and we would not recommend deploying the predictor for this purpose.

Ordering by predicted output length alone is worse. On conversation, where the predictor has some signal, SJF-Pred matches the others in $\bar{L}_n$ but doubles P_{99} (207 s against 103–104 s). On code, where it is near chance, SJF-Pred attains $\bar{L}_n = 23081$ against 16724 for FCFS, with P_{99} of 1882 s against 271 s. Optimising a quantity that cannot be estimated adds ordering noise without compensating benefit, and a static priority from a near-random key starves whatever it ranks last. CB-SJF-Work avoids this because the observable prefill term anchors its ranking. This negative result is scoped to the content-blind setting: it does not evaluate the prompt-reading predictors of S^3 [3] or Fu *et al.* [4], which we did not implement and which use a signal ours cannot see.

D. SLO attainment

CB-SJF-Work roughly doubles SLO attainment over FCFS, 34.5% against 18.8% (conversation) and 15.4% against 8.6% (code), matching Oracle-Work to within 0.1 points; SJF-Ctx matches both, so on this metric the entire gain comes from ordering by context length. Absolute attainment is low, and not because the replica is saturated: at our lowest load it is still 39.2% and 17.6%. A single unchunked prefill step for a multi-thousand-token prompt consumes much of the 2 s time-to-first-token budget before any queueing, so the target is unreachable at any load in this engine configuration. The comparison between policies remains meaningful; the absolute level should not be read as a claim about production SLO attainment.

VII. LIMITATIONS AND THREATS TO VALIDITY

The cost model is analytical. We do not have access to the accelerator hardware these traces were served on, and our coefficients are modelled rather than measured. The quantity our mechanism depends on, however, is the ratio of context to output tokens, which is a property of the traces: 15.5:1 and 110.7:1 in aggregate. We therefore stress that ratio directly, by scaling output length, rather than perturbing a cost coefficient. Scaling output length moves both the decode share of work

TABLE I

SCHEDULING PERFORMANCE AT OFFERED LOAD $\rho = 0.92$, AVERAGED OVER 20 (CONVERSATION) AND 11 (CODE) DISJOINT WINDOWS SAMPLED ACROSS THE FULL WEEK (MEAN $\pm$ S.D.). $\bar{L}_n$ IS MEAN NORMALISED LATENCY (MS PER OUTPUT TOKEN); P_{99} IS 99TH-PERCENTILE END-TO-END LATENCY (S); SLO IS THE FRACTION OF REQUESTS MEETING BOTH THE TTFT AND TBT TARGETS. NOTE THAT EVERY WORK- OR LENGTH-AWARE POLICY, INCLUDING BOTH ORACLES, DEGRADES P_{99} RELATIVE TO FCFS: ORDERING SHORT REQUESTS FIRST DEFERS LONG ONES BY CONSTRUCTION. [†]ORACLE POLICIES REQUIRE KNOWLEDGE UNAVAILABLE AT ADMISSION TIME AND ARE UPPER BOUNDS, NOT DEPLOYABLE SYSTEMS.

Policy	Conversation			Code		
	$\bar{L}_n$	P_{99} (s)	SLO (%)	$\bar{L}_n$	P_{99} (s)	SLO (%)
FCFS (engine default)	1161 $\pm$ 921	104	18.8	16724 $\pm$ 16657	271	8.6
SJF-Ctx	505 $\pm$ 254	103	34.5	6337 $\pm$ 5519	562	15.4
SJF-Pred	529 $\pm$ 240	207	33.0	23081 $\pm$ 28340	1882	12.2
CB-SJF-Work (ours)	506 $\pm$ 253	104	34.5	6327 $\pm$ 5542	556	15.4
Oracle-SJF [†]	287 $\pm$ 23	129	31.9	2289 $\pm$ 669	1261	14.4
Oracle-Work [†]	436 $\pm$ 184	102	34.4	3676 $\pm$ 2769	528	15.3

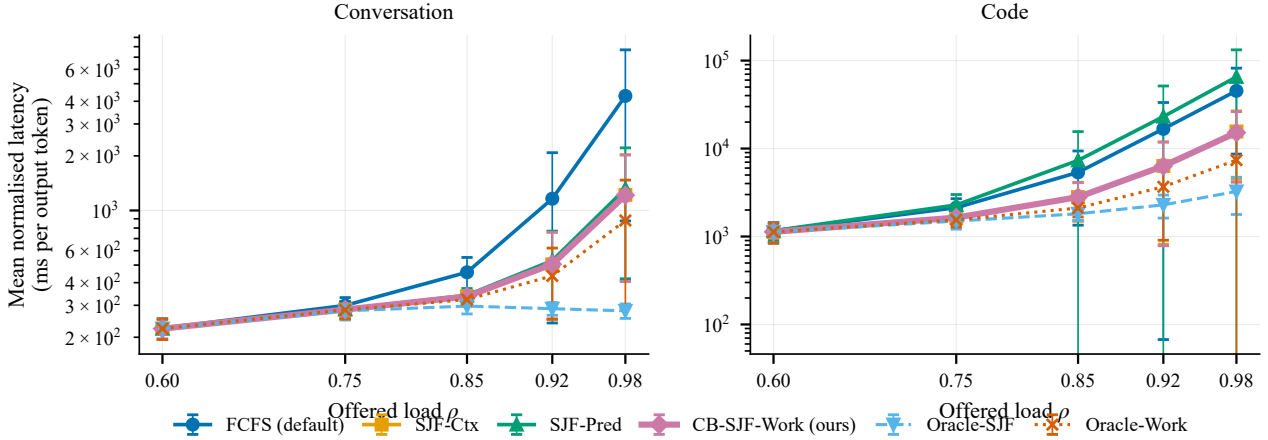


Fig. 1. Mean normalised latency against offered load, over 20 (conversation) and 11 (code) windows sampled across a full production week (error bars: s.d.). Below $\rho \approx 0.75$ the queue is shallow and all policies nearly coincide. CB-SJF-Work tracks Oracle-Work closely and is visually indistinguishable from SJF-Ctx; SJF-Pred, which orders by predicted output length, diverges and on the code workload is worse than FCFS.

TABLE II

CB-SJF-WORK RELATIVE TO FCFS AGAINST OFFERED LOAD. ROWS PER WORKLOAD: MEAN-LATENCY IMPROVEMENT (%); ITS PER-WINDOW RANGE (%); PERCENTAGE OF THE ORACLE-ATTAINABLE IMPROVEMENT RECOVERED; AND THE RATIO OF 99TH-PERCENTILE LATENCY TO FCFS, WHERE A VALUE ABOVE 1.0 MEANS THE TAIL IS WORSE. THE FIRST THREE ROWS ARE THE BENEFIT, THE FOURTH IS THE COST.

Workload	Quantity	Offered load ρ				
		0.60	0.75	0.85	0.92	0.98
Conv.	improvement	0.4	5.1	26.4	56.5	71.7
	range	-0-2	1-14	6-44	16-69	39-77
	oracle gap	96.3	92.4	90.4	90.4	90.2
	$P_{99}/FCFS$	1.00	1.00	0.98	1.00	1.86
Code.	improvement	2.7	22.9	47.7	62.2	66.5
	range	1-12	4-43	14-61	27-71	53-73
	oracle gap	93.8	83.8	78.6	79.7	79.4
	$P_{99}/FCFS$	1.01	1.05	1.33	1.70	3.12

and the KV footprint, which is the binding constraint in our engine; scaling a decode cost coefficient alone leaves the KV footprint untouched and is uninformative.

Longer generations, including reasoning and agentic

workloads. Scaling output length by up to $16\times$ (Table IIIa) drives the prefill share of engine time from 75.4% to 11.2% on conversation. The improvement over FCFS does not collapse; it rises, from 46.0% to 75.1%, because ordering by observable work stays a good ordering even when the observable term is no longer the majority. What degrades is the price of content-blindness: CB-SJF-Work runs $1.08\times$ the mean normalised latency of Oracle-Work at $1\times$ but $3.76\times$ at $16\times$ (conversation) and $5.53\times$ (code). Correspondingly the learned predictor, worth 0.3% on today’s traces, becomes worth 21.1%. For long-generation workloads our conclusions would need revisiting and prompt-based prediction would likely repay its cost.

Prefix caching. This is the most serious threat and the traces cannot answer it. Production stacks serve repeated prefixes from cache, and multi-turn conversation is the case with the highest hit rates, but the traces record context length and not cache residency. Reducing executed prefill to model a hit rate, while the scheduler still observes the full context length (cache-hit length depends on token identity and so is not content-blind), the improvement over FCFS falls to 12.8% (conversation) and 14.8% (code) at a 90% hit rate (Table IIIb). The recovered oracle gap stays high, 85.5% and

TABLE III

ROBUSTNESS AT $\rho = 0.92$. TOP: SCALING OUTPUT LENGTH, WHICH DRIVES THE PREFILL SHARE OF ENGINE TIME DOWN AND MODELS LONGER-GENERATION WORKLOADS. THE IMPROVEMENT OVER FCFS SURVIVES, BUT THE PENALTY FOR CONTENT-BLINDNESS (CB/ORACLE-WORK RATIO) GROWS AND THE LEARNED PREDICTOR BECOMES WORTHWHILE. BOTTOM: MODELLING A PREFIX-CACHE HIT RATE, WHICH SHRINKS THE OBSERVABLE WORK AND SUBSTANTIALLY REDUCES THE BENEFIT OF ANY WORK-AWARE ORDERING.

(a) Output-length scaling					
	1×	2×	4×	8×	16×
Conversation	75	62	44	25	11
	46	50	57	65	75
	1.08	1.24	1.62	2.51	3.76
	0.3	0.4	1.3	6.3	21.1
Code	93	88	81	69	49
	63	62	62	59	50
	1.70	1.86	2.14	2.63	5.53
	0.4	1.0	4.1	6.4	10.7
(b) Prefix-cache hit rate					
	0%	50%	75%	90%	
Conversation	46	36	25	13	
Code	63	59	49	15	

(a) Rows per workload: prefill share of engine time (%); improvement over FCFS (%); CB-SJF-Work/Oracle-Work latency ratio (×); predictor gain over SJF-Ctx (%). (b) Improvement over FCFS (%).

74.5%, because caching shrinks the benefit available to any scheduler rather than to ours specifically, but the absolute case for work-aware admission weakens considerably. Prefix-cache-aware content-blind scheduling is the main open problem this work exposes.

Non-preemptive admission. Our scheduler controls admission order only: no preemption, swapping or aging. This isolates the effect of admission information, and it is also why our tails degrade; preemptive schedulers [9], [10] address that failure mode. Our oracles are upper bounds on admission-order policies specifically. We likewise model a single replica without chunked prefill [6] or phase disaggregation [7], [8]. Chunking does not change total prefill work or KV occupancy, which is what binds here, so we expect the central claim to survive, but we have not verified it. Finally, two workloads from one provider over one week support a claim about the regime we measured, not a universal one.

Reproducibility. All experiments use public data. Windows are selected deterministically, the pipeline is content-hashed to verify byte-identical reruns, seeds live in one configuration module, and every number in this paper is generated programmatically from stored result files rather than transcribed. Code will be released upon acceptance.

VIII. CONCLUSION

We asked what LLM inference scheduling loses if the scheduler may not read prompt content. On 44.1M requests of Azure production traffic the answer is: less than the literature’s focus on prompt-based length prediction would suggest. Prefill, the component observable without reading content, is the larger part of service work on these traces, and ordering by estimated

total work recovers 79.3% to 90.3% of the oracle-attainable improvement in mean normalised latency and roughly doubles SLO attainment, even on a workload where output length is statistically unpredictable from metadata.

Three qualifications belong with that result. The gain is paid for in the tail, by a factor of up to $3.1\times$, and an oracle scheduler pays the same price, so it is a property of shortest-first admission rather than of content-blindness. A learned output-length predictor is not worth deploying on today’s traces, contributing 0.1% to 1.3% over ordering by raw context length, though it becomes worth 21% once generations are $16\times$ longer. And prefix caching, which these traces cannot expose, would shrink the benefit for every policy studied here.

The practical implication is narrow but useful: on workloads resembling these, a scheduler that never sees prompt content gives up little, and ordering by context length alone captures nearly all of it. The natural next steps are to combine content-blind ordering with preemption or aging, which our tail results show is needed, and to extend it to prefix-cache-aware serving.

REFERENCES

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [2] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *Proc. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [3] Y. Jin, C.-F. Wu, D. Brooks, and G.-Y. Wei, “S³: Increasing GPU utilization during generative inference for higher throughput,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [4] Y. Fu, S. Zhu, R. Su, A. Qiao, I. Stoica, and H. Zhang, “Efficient LLM scheduling by learning to rank,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [5] Microsoft Azure, “AzurePublicDataset: Azure LLM inference trace 2024,” <https://github.com/Azure/AzurePublicDataset>, 2024, accessed 2026-08-02.
- [6] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, “Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve,” in *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [7] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” in *Proc. 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- [8] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [9] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin, “FastServe: Iteration-level preemptive scheduling for large language model inference,” in *Proc. 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026.
- [10] Z. Zhang and H. Shen, “PecSched: Preemptive and efficient cluster scheduling for LLM inference,” arXiv:2409.15104, 2024.
- [11] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proc. 26th ACM Symposium on Operating Systems Principles (SOSP)*, 2017.
- [12] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, “DynamoLLM: Designing LLM inference clusters for performance and energy efficiency,” in *Proc. IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025.